Name: Arjun B R

Intern ID: IN126049802

Title: RAG-Based Customer Support Assistant
Technical Documentation

1. Introduction

➢ **What is RAG?**

Retrieval-Augmented Generation (RAG) is an approach that combines two key ideas: retrieving relevant information from a data source and then generating an answer using a language model. Instead of relying only on pre-trained knowledge, the system first searches for relevant content and then uses that content to produce accurate responses.

➢ **Why it is Needed?**

Traditional chatbots often generate incorrect or vague answers because they lack access to real-time or domain-specific data. This leads to hallucination, where the model creates information that is not actually present.

RAG solves this problem by grounding the response in actual document content. This improves accuracy, reliability, and trust in the system.

Use Case Overview

This system is designed as a customer support assistant that answers questions based on PDF documents uploaded by the user. It can be used in scenarios such as:

- Policy document explanation
- Customer service automation
- Agreement and contract analysis
- Knowledge base querying

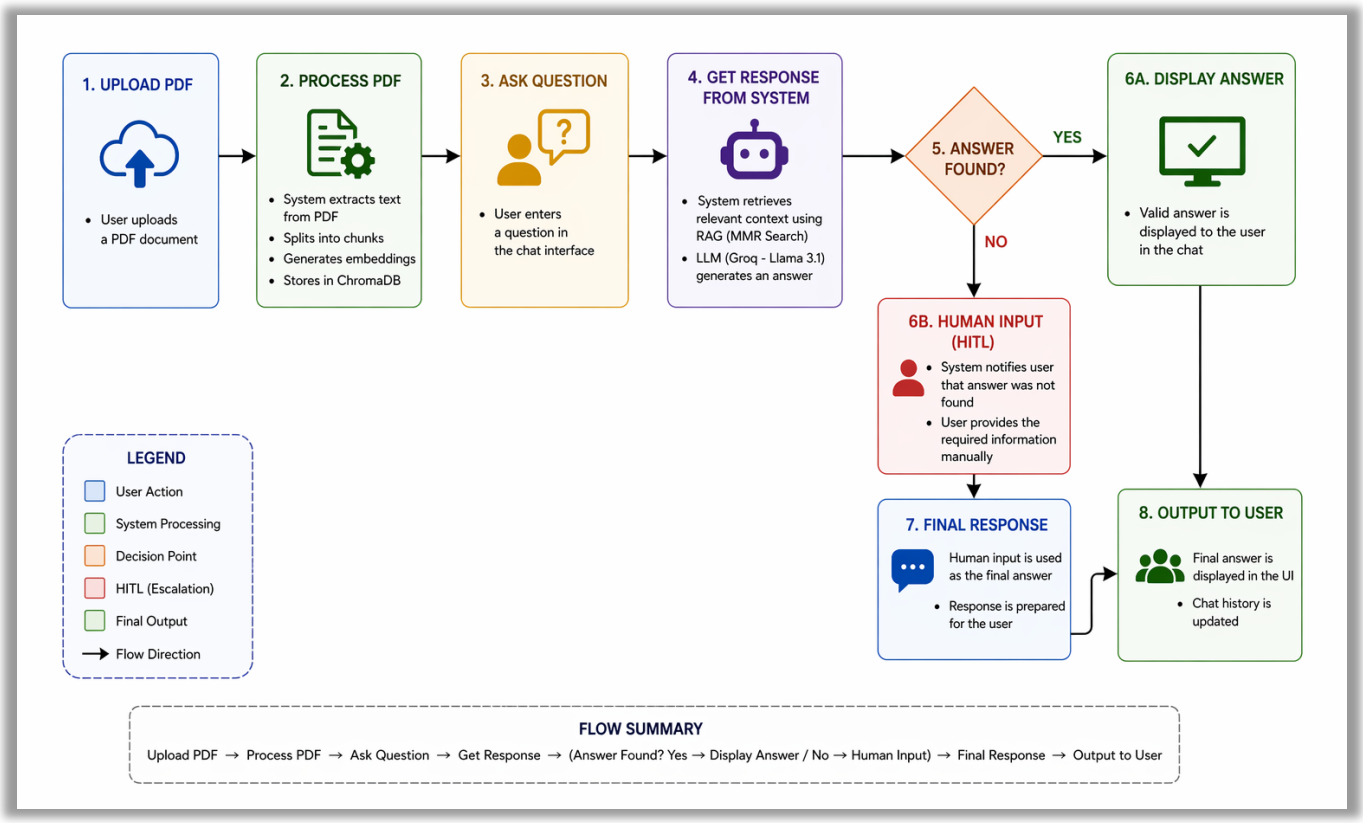

Fig: UI Flow Diagram

This diagram shows how the user interacts with the system. The process starts with uploading a PDF, followed by asking a query. The system processes the query and either generates an answer or requests human input through the HITL mechanism.

2. System Architecture Explanation

- The system follows a modular pipeline where each component performs a specific role in processing data and generating responses.

- The user interacts with the system through a web interface built using Streamlit. The user uploads a PDF file, which is processed by the document ingestion pipeline. The extracted text is divided into smaller chunks and converted into embeddings.

- These embeddings are stored in ChromaDB, which allows fast similarity-based retrieval. When a user asks a question, the system retrieves the most relevant chunks and sends them to the language model (Groq API).

- LangGraph is used to control the workflow. It decides whether to generate an answer or escalate to human input through the HITL module.

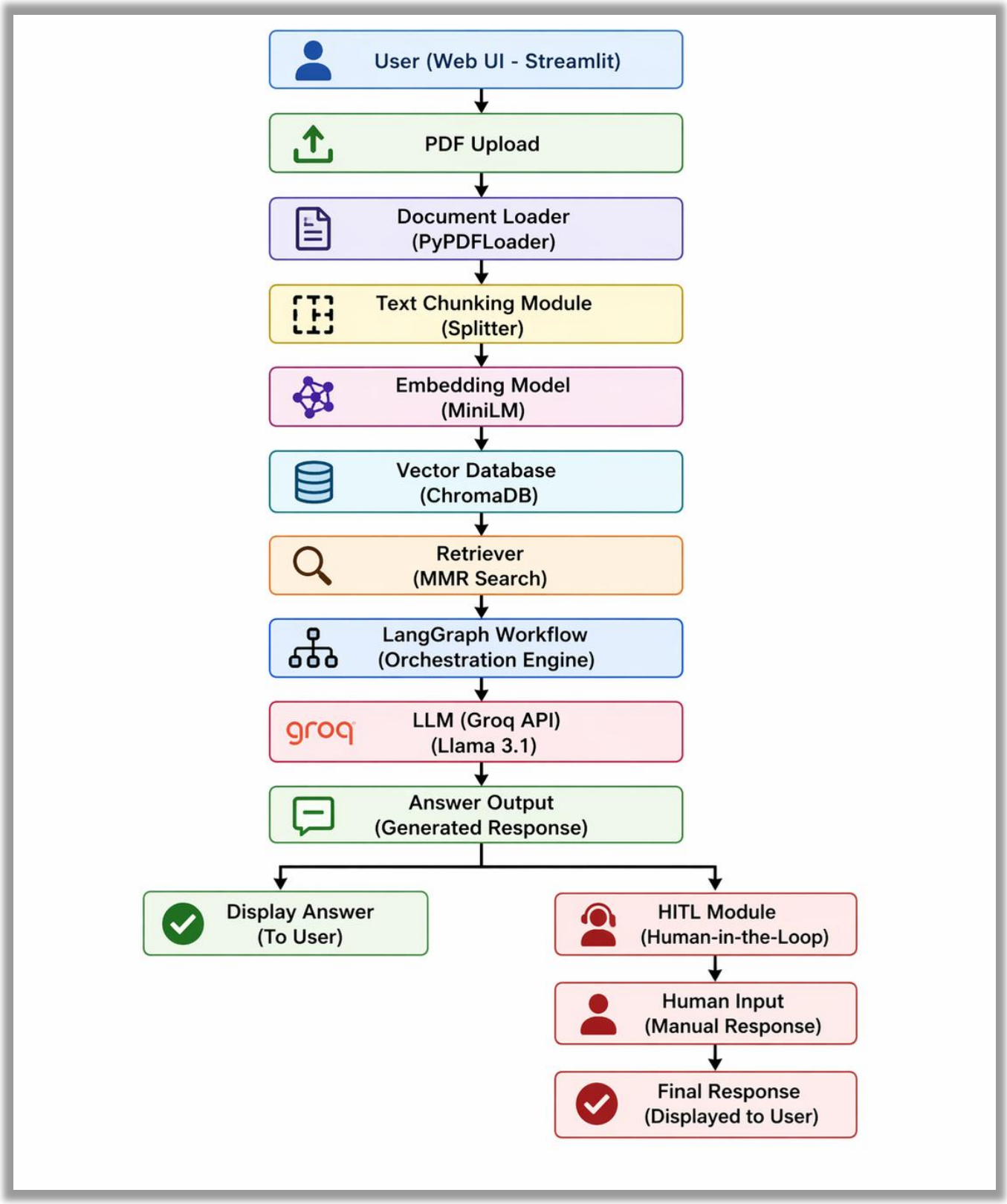

Fig: System Architecture Diagram

This diagram represents the overall architecture of the system, including document processing, embedding generation, retrieval, LLM processing, and HITL integration.

Component Interactions

- The UI collects user input and PDF
- The document loader extracts text
- The chunking module splits text
- The embedding model converts text to vectors
- ChromaDB stores and retrieves embeddings
- The retriever selects relevant chunks
- The LLM generates the final answer
- LangGraph controls the decision flow
- HITL handles fallback cases

3. Design Decisions

- **Chunk Size Choice**

  The system uses a chunk size of around 700 characters with overlap. This ensures that important information is not split across chunks and improves retrieval accuracy.

- **Embedding Strategy**

  MiniLM is used as the embedding model because it provides a good balance between performance and computational efficiency. It captures semantic meaning effectively while remaining lightweight.

- **Retrieval Approach**

  Max Marginal Relevance (MMR) is used instead of simple similarity search. This helps in retrieving diverse and relevant chunks, reducing redundancy and improving answer quality.

Prompt Design Logic

The prompt is carefully designed to ensure that the model:

- Answers strictly from context
- Does not generate unsupported information
- Returns “NOT FOUND” when data is missing

This prevents hallucination and ensures reliability.

4. Workflow Explanation

LangGraph Usage

LangGraph is used to define a structured workflow for the system. It allows clear separation of processing steps and supports conditional routing.

Node Responsibilities

- **Processing Node**
  - Retrieves relevant chunks from the database
  - Prepares context
  - Sends query + context to LLM
- **Output Node**
  - Returns final answer
  - Or triggers HITL if answer is not found
  - State Transitions

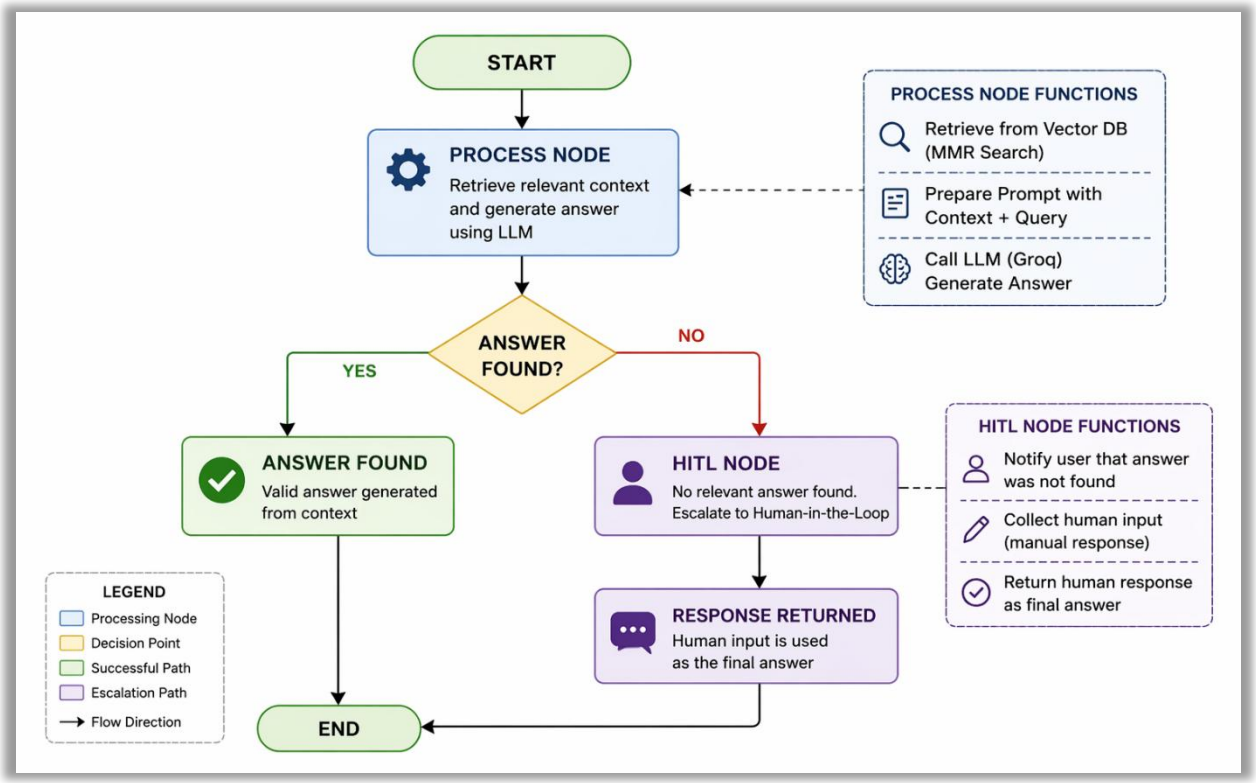

Fig: LangGraph Workflow Diagram

This diagram illustrates how the system processes queries using LangGraph, including decision-making and routing to HITL when required.

The system maintains a state object containing:

- Query
- Retrieved context
- Generated answer

This state is updated at each stage and passed between nodes to ensure proper workflow execution.

5. Conditional Logic

Intent Detection

The system determines whether a query can be answered based on the availability of relevant context. It does not rely on predefined intents but uses semantic matching.

Routing Decisions

The system follows two main paths:

- If answer is found → return response
- If answer is not found → trigger HITL
- Low Confidence Handling

If the retrieved context is weak or unclear, the system avoids generating an answer and escalates to human input.

Missing Context

If no relevant chunks are retrieved, the system does not attempt to guess and directly triggers HITL.

Complex Query Handling

For multi-part or ambiguous queries, the system may not retrieve sufficient context. In such cases, escalation ensures correctness.

Ambiguous Query Handling

If the user query is unclear, vague, or lacks sufficient detail, the system may retrieve irrelevant or incomplete context. In such cases, instead of generating a potentially incorrect answer, the system avoids making assumptions and escalates the query to the HITL module. This ensures that the final response remains accurate and reliable.

6. HITL Implementation

Role of Human Intervention

- The Human-in-the-Loop (HITL) mechanism plays a critical role in ensuring the reliability of the system. While the RAG pipeline is designed to retrieve and generate accurate answers, there are situations where the system may not find sufficient information in the document.

- In such cases, instead of generating an incorrect or misleading response, the system triggers HITL. This allows a human user to manually provide the correct answer. This approach ensures that the system maintains high accuracy and avoids hallucination.

- HITL acts as a safety layer that improves trust in the system, especially in real-world customer support scenarios where incorrect answers can lead to serious issues.

Benefits

- Improves accuracy by avoiding incorrect answers
- Provides a fallback for unknown or complex queries
- Builds trust in the system
- Ensures better handling of edge cases

Limitations

- Requires manual effort from the user
- Slows down the response process
- Not fully automated
- Cannot scale efficiently without additional automation

7. Challenges & Trade-offs

❖ **Retrieval Accuracy vs Speed**

There is a trade-off between retrieving more data and maintaining system speed. Increasing the number of retrieved chunks improves the chances of finding relevant information, but it also increases processing time and latency. Therefore, a balance is required to ensure both accuracy and performance.

❖ **Chunk Size vs Context Quality**

Choosing the right chunk size is important. Larger chunks provide more context but may include irrelevant information, which can confuse the model. Smaller chunks improve precision but may miss important details. The system uses overlapping chunks to balance this trade-off.

❖ **Cost vs Performance**

Using advanced language models improves response quality but increases computational cost. Similarly, increasing retrieval depth improves accuracy but adds processing overhead. The system is designed to balance performance and cost by using efficient models and optimized retrieval methods.

Additional Practical Challenges

- Handling poorly formatted PDFs
- Extracting text from scanned documents
- Ensuring consistent retrieval across different document types
- Managing API latency during peak usage

8. Testing Strategy

❖ **Testing Approach**

The system is tested using a variety of query types to evaluate its performance and reliability. Testing focuses on ensuring that the system retrieves correct information and handles edge cases properly.

Different types of tests include:

- Functional testing (checking correctness of answers)
- Edge case testing (handling unknown queries)
- Performance testing (response time and latency)

❖ **Sample Queries**

Some example queries used during testing include:

- What is refund policy?
- Who can apply for refund?

- What is GST?
- Explain CGST
- What is not covered in this document?

❖ **Expected Results**

- For relevant queries → accurate, context-based answers
- For partially relevant queries → limited but correct responses
- For irrelevant queries → HITL should be triggered

❖ **Evaluation Criteria**

The system is evaluated based on:

- Accuracy of answers
- Response time
- Correct HITL triggering
- Consistency of results

9. Future Enhancements

- **Multi-Document Support**

  Extend the system to support multiple PDF documents simultaneously.
- **Feedback Loop**

  Store human responses to improve future performance and reduce dependency on HITL.
- **Memory Integration**

  Add conversational memory to handle follow-up questions and context continuity.
- **Deployment**

  Deploy the system on cloud platforms for scalability and real-world usage.
- **Additional Enhancements**
  - Voice-based input and output
  - Better UI design for user experience
  - Support for multiple languages
  - Integration with enterprise systems

Conclusion

The RAG-based Customer Support Assistant demonstrates how combining retrieval mechanisms with language models can significantly improve the accuracy and reliability of automated responses. By grounding answers in document-based context, the system avoids hallucination and ensures meaningful outputs.

The integration of LangGraph enables structured workflow execution, while the Human-in-the-Loop mechanism provides an additional layer of reliability by handling cases where the system cannot generate confident answers.

Overall, the system successfully achieves its objective of providing a context-aware, scalable, and practical solution for document-based question answering. It can be further enhanced with additional features such as multi-document support, memory integration, and cloud deployment for real-world applications.